Agencez le contenu de vos pages
(deuxième partie)

Mathieu Nebra

Mis à jour le 18/06/2024

1 Faites votre mise en page avec Flexbox

1.1 Comprenez la logique : un conteneur, des éléments

Pour faire de la mise en page avec Flexbox, il faut définir un conteneur, et placer à l'intérieur plusieurs éléments.

Imaginez un carton dans lequel vous rangez plusieurs objets : c'est le principe !

Sur une même page web, vous pouvez avoir plusieurs conteneurs (plusieurs cartons, si vous préférez). Vous pouvez en créer autant que nécessaire pour obtenir la mise en page que vous voulez.

Commençons par étudier le fonctionnement d'un carton (euh pardon, d'un conteneur).



Un conteneur (container en anglais) est une balise qui peut renfermer d'autres balises, comme du texte ou encore des images. Les conteneurs les plus célèbres sont les balises `<div>` et `<span>`.

```
<div class="container">
<div class="element element1">Élément 1</div>
<div class="element element2">Élément 2</div>
<div class="element element3">Élément 3</div>
</div>
```

Par défaut, mes éléments vont se mettre les uns en dessous des autres, ce sont des blocs. C'est le comportement normal dont nous avons l'habitude. Découvrons donc maintenant l'intérêt de Flexbox !

1.2 Alignez les éléments d'un conteneur avec display : flex ;

Il suffit d'une seule propriété CSS, et tout change ! Cette propriété, c'est flex et on l'applique au conteneur :

```css
.container {
    display: flex;
}
```

... alors les blocs se placent par défaut côte à côte. Magique !



1.3 Donnez leur une direction avec la propriété flex-direction

Flexbox permet d'agencer ces éléments dans le sens que l'on veut. Avec `flex-direction`, on peut les positionner verticalement ou encore les inverser. Cette propriété CSS peut prendre les valeurs suivantes :

- `row` : organisés sur une ligne (par défaut) ;
- `column` : organisés sur une colonne ;
- `row-reverse` : organisés sur une ligne, mais en ordre inversé ;
- `column-reverse` : organisés sur une colonne, mais en ordre inversé.

Ici, on fait simplement :

```css
.container {
    display: flex;
    flex-direction: column;
}
```

Ce qui donne :



Les éléments du conteneur sont disposés en colonne



Mais mais... c'est pareil qu'au début, non ? On avait ce résultat sans Flexbox, après tout !

C'est vrai. Mais maintenant que nos éléments sont flex, ils ont tout un tas d'autres propriétés utiles que nous allons voir juste après, on va y revenir.

Essayez aussi de tester l'ordre inversé, pour voir :

```
.container {
    display: flex;
    flex-direction: column-reverse;
}
```

☞



Les éléments sont en colonne... dans l'ordre inverse !

Regardez bien la différence : les blocs sont maintenant dans l'ordre inverse ! Je n'ai pas du tout changé le code HTML, qui reste le même depuis le début.

1.4 Retournez à la ligne avec la propriété flex-wrap

Par défaut, les blocs essaient de rester sur la même ligne s'ils n'ont pas la place, quitte à "s'écraser", et provoquer parfois des anomalies dans la mise en page (certains éléments pouvant dépasser de leur conteneur). Si vous voulez, vous pouvez demander à ce que les blocs aillent à la ligne lorsqu'ils n'ont plus la place, avec `flex-wrap`.

Voilà les différentes valeurs de flex-wrap :

- `nowrap` : pas de retour à la ligne (par défaut) ;
- `wrap` : les éléments vont à la ligne lorsqu'il n'y a plus la place ;
- `wrap-reverse` : les éléments vont à la ligne, lorsqu'il n'y a plus la place, en sens inverse.

```
.container {
    display: flex;
    flex-wrap: nowrap;
    /* OU wrap;
    OU wrap-reverse; */
}
```

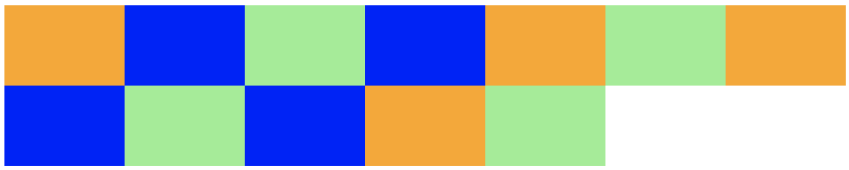
Dans les illustrations suivantes, vous pouvez voir les différents comportements de nos blocs en fonction de la valeur que l'on attribue à `flex-wrap` :

Avec `flex-wrap: nowrap;` :



Les éléments se serrent pour tenir sur la même ligne

Avec `flex-wrap: wrap;` :



Les éléments passent à la ligne s'ils ne rentrent pas tous sur la même ligne



Cette propriété sera particulièrement utile quand nous aurons besoin d'adapter nos pages aux plus petits écrans. Nous verrons ça dans la prochaine partie de ce cours.

Avec `flex-wrap: wrap-reverse;` :



Les éléments passent à la ligne mais à l'envers

1.5 Alignez les éléments sur un axe principal et secondaire

Les éléments sont organisés par défaut de manière horizontale. Mais ils peuvent être organisés de manière verticale. Selon le choix que vous faîtes, ça va définir ce qu'on appelle **l'axe principal**. Il y a aussi un axe secondaire :

- si vos éléments sont organisés horizontalement, l'axe secondaire est l'axe vertical ;
- si vos éléments sont organisés verticalement, l'axe secondaire est l'axe horizontal.

Pourquoi je vous raconte ça ? Parce que nous allons découvrir comment aligner nos éléments sur l'axe principal **et** sur l'axe secondaire.

1.5.1 Alignez sur l'axe principal avec la propriété justify-content



Pour faire simple, partons sur des éléments organisés horizontalement (c'est le cas par défaut).

Pour changer leur alignement, on va utiliser `justify-content`, qui peut prendre ces valeurs :

- `flex-start` : alignés au début (par défaut) ;
- `flex-end` : alignés à la fin ;
- `center` : alignés au centre ;
- `space-between` : les éléments sont étirés sur tout l'axe (il y a de l'espace entre eux) ;
- `space-around` : idem, les éléments sont étirés sur tout l'axe, mais ils laissent aussi de l'espace sur les extrémités.

On écrit alors :

```
.container {
    display: flex;
    justify-content: flex-start /* OU flex-end
    OU center
    OU space-between
    OU space-around; */
}
```

Maintenant, voici ce qu'il faut bien comprendre : **ça marche aussi si vos éléments sont dans une direction verticale**. Dans ce cas, l'axe vertical devient l'axe principal, et `justify-content` s'applique aussi :

```
.container {
    display: flex;
    flex-direction: column;
    justify-content: center;
    /* Un peu de hauteur pour que les éléments aient la place de bouger :*/
    height: 350px;
}
```

Avec une direction verticale ("column"), le centrage fonctionne de la même façon, mais cette fois en hauteur.

1.5.2 Alignez sur l'axe secondaire avec align-items

Si nos éléments sont placés dans une direction horizontale (ligne), l'axe secondaire est... vertical. Et inversement : si nos éléments sont dans une direction verticale (colonne), l'axe secondaire est horizontal.

La propriété align-items permet de changer leur alignement sur l'axe secondaire, grâce aux valeurs :

- `stretch` : les éléments sont étirés sur tout l'axe (valeur par défaut) ;
- `flex-start` : alignés au début ;
- `flex-end` : alignés à la fin ;
- `center` : alignés au centre ;
- `baseline` : alignés sur la ligne de base (semblable à `flex-start`).

Exemple :

```
.container {
    display: flex;
    justify-content: center;
    align-items: center;
}
```

Saint Graal du développeur web, le centrage vertical et horizontal peut d'ailleurs être obtenu encore plus facilement. Dites-vous que votre conteneur est une Flexbox, et établissez des marges automatiques sur les éléments à l'intérieur :

```
.container {
    display: flex;
}

.element {
    margin: auto;
}
```

1.6 Répartissez les blocs sur plusieurs lignes avec align-content

Si vous avez plusieurs lignes dans votre Flexbox, vous pouvez choisir comment celles-ci seront réparties avec `align-content`.



Cette propriété n'a aucun effet s'il n'y a qu'une seule ligne dans la Flexbox.

Prenons donc un cas de figure où nous avons plusieurs lignes. J'autorise les éléments à aller à la ligne avec `flex-wrap`.

Voyons voir comment les lignes se répartissent différemment avec la nouvelle propriété `align-content` que je voulais vous présenter. Elle peut prendre ces valeurs :

- `stretch` (par défaut) : les éléments s'étirent pour occuper tout l'espace ;
- `flex-start` : les éléments sont placés au début ;
- `flex-end` : les éléments sont placés à la fin ;
- `center` : les éléments sont placés au centre ;
- `space-between` : les éléments sont séparés avec de l'espace entre eux ;
- `space-around` : idem, mais il y a aussi de l'espace au début et à la fin.



Et si vous voulez vous entraîner pour vous assurer que vous maîtrisez bien les propriétés de Flexbox, je vous recommande le jeu Flexbox Froggy. Avec les différentes propriétés, vous devrez replacer les grenouilles sur les nénuphars.

1.7 Travail à faire

Maintenant que vous maîtrisez les propriétés de Flexbox, nous allons en profiter pour mettre en forme les pages du portfolio de Robbie Lens !



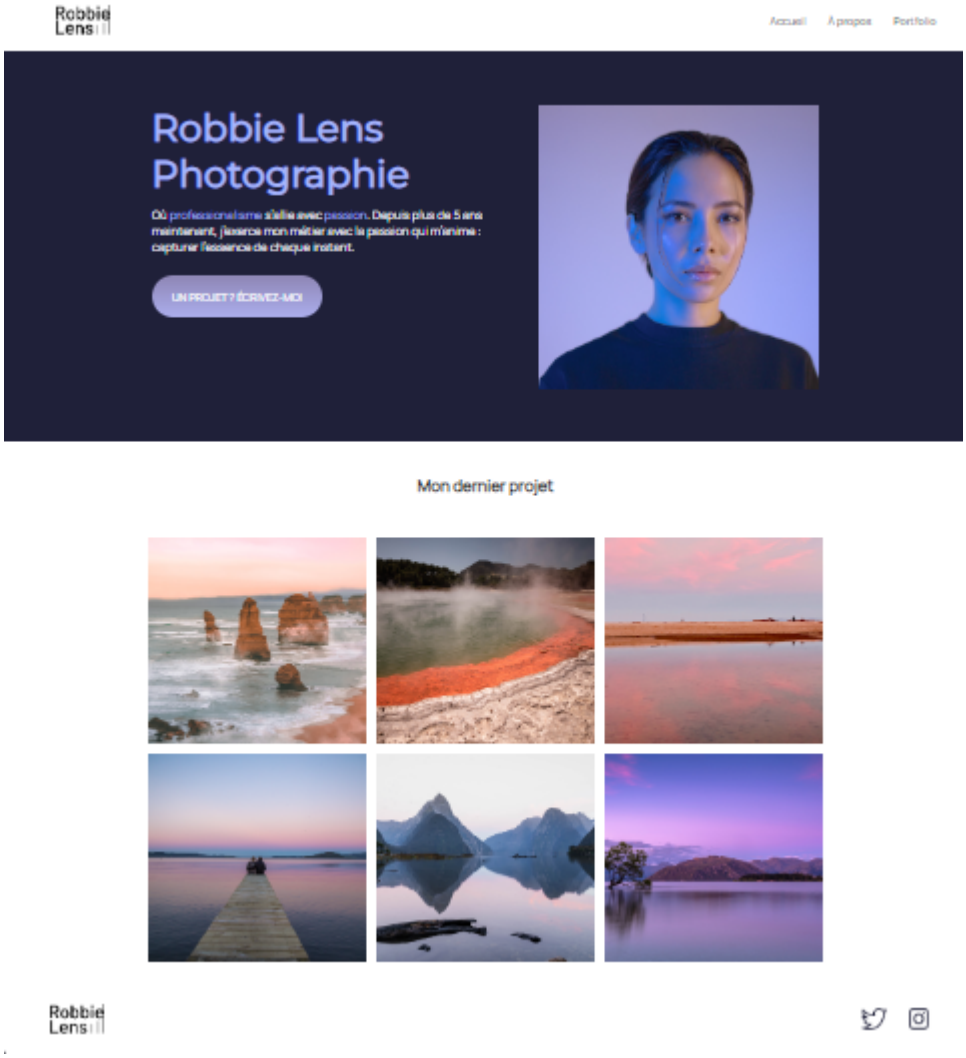
Des images ont été ajoutées dans le dossier "images" et des sections ont été ajoutées dans la page d'accueil.



Vous allez :

- utiliser `flex` pour avoir le logo en haut à gauche de la page, et les liens de navigation à droite dans l'en-tête ;
- modifier le style du footer pour avoir le logo en haut à gauche de la page, et les liens de navigation à droite ;
- réagencer les éléments pour avoir le titre de la page d'accueil, le paragraphe, et le lien ayant la classe "CTA" à gauche, et la photo à droite ;
- importer les images dans la page d'accueil, et créer une grille d'images avec deux rangées de 3 images chacune.

Voici un aperçu du résultat attendu :



1.8 Synthèse

- Le principe de Flexbox est d'avoir un conteneur avec plusieurs éléments à l'intérieur. Avec `display: flex;` sur le conteneur, les éléments à l'intérieur sont agencés en mode Flexbox (horizontalement, par défaut).

- Flexbox peut gérer toutes les directions. Avec `flex-direction`, on peut indiquer si les éléments sont agencés horizontalement (par défaut) ou verticalement. Cela définit ce qu'on appelle l'axe principal.

- L'alignement des éléments se fait sur l'axe principal avec `justify-content` et sur l'axe secondaire avec `align-items`.

- Avec `flex-wrap`, on peut autoriser les éléments à revenir à la ligne s'ils n'ont plus d'espace.

- S'il y a plusieurs lignes, on peut indiquer comment les lignes doivent se répartir entre elles avec `align-content`.

2 Découvrez les bases de CSS Grids

2.1 Définissez une grid avec la propriété CSS display : grid ;

De la même manière que pour Flexbox, lorsque vous utilisez CSS Grids, vous pouvez imaginer un conteneur, une sorte de "carton" dans lequel vous allez mettre les éléments de votre grid. Sauf que, cette fois-ci, vous allez avoir besoin de déclarer la propriété `display: grid;` :

```
.conteneur {
    display: grid;
}
```

Ensuite, il faut préciser quelques éléments supplémentaires si on veut que ça fonctionne. Pour ce faire, il s'agit de se demander comment représenter nos éléments. Lorsque vous créez un tableau, vous réfléchissez aux colonnes et aux rangées qu'il faudra créer. Eh bien, c'est la même chose ici !

On va définir les colonnes et les rangées avec deux propriétés :

- `grid-template-columns` pour le nombre de colonnes, et la largeur de chacune d'entre elles.
- `grid-template-rows` pour le nombre de rangées, et la hauteur de chacune d'entre elles.

2.1.1 Définissez vos colonnes avec la propriété CSS grid-template-columns

On a déjà 9 éléments déclarés côté HTML de la manière suivante :

```
<div class="conteneur">
    <div class="box">Élément 1</div>
    <div class="box">Élément 2</div>
    <div class="box">Élément 3</div>
    <div class="box">Élément 4</div>
    <div class="box">Élément 5</div>
    <div class="box">Élément 6</div>
    <div class="box">Élément 7</div>
    <div class="box">Élément 8</div>
    <div class="box">Élément 9</div>
</div>
```

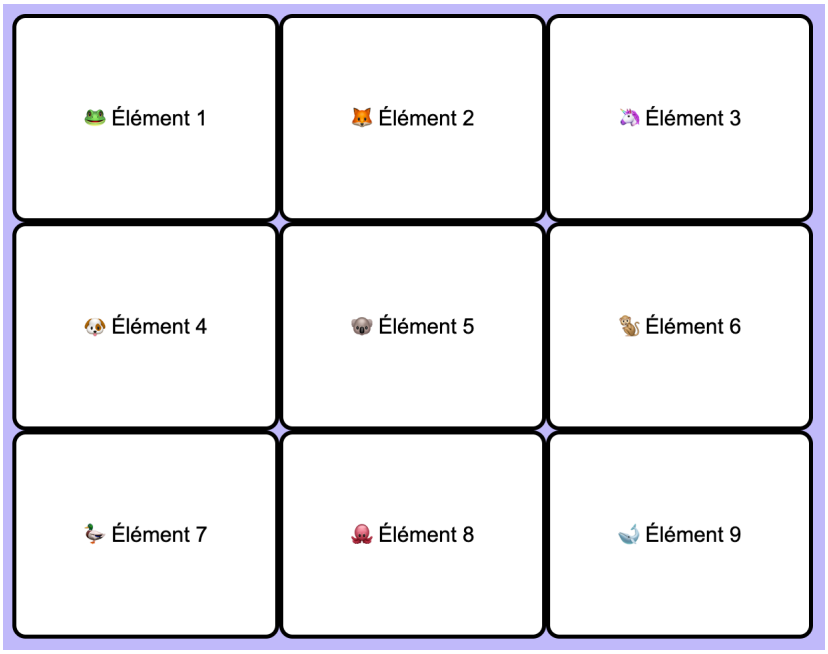
La classe CSS `.box` permet d'ajouter un peu de style aux éléments. Pour que le contenu soit plus harmonieux, on va donner à chaque élément une hauteur de `150px` :

```
.box {
  height: 150px;
}
```

Plus bas, dans le fichier CSS, créons 3 colonnes, chacune de `200px` de large, à l'aide de la propriété `grid-template-columns` :

```css
.conteneur {
    display: grid;
    grid-template-columns: 200px 200px 200px;
}
```

Et on obtient :



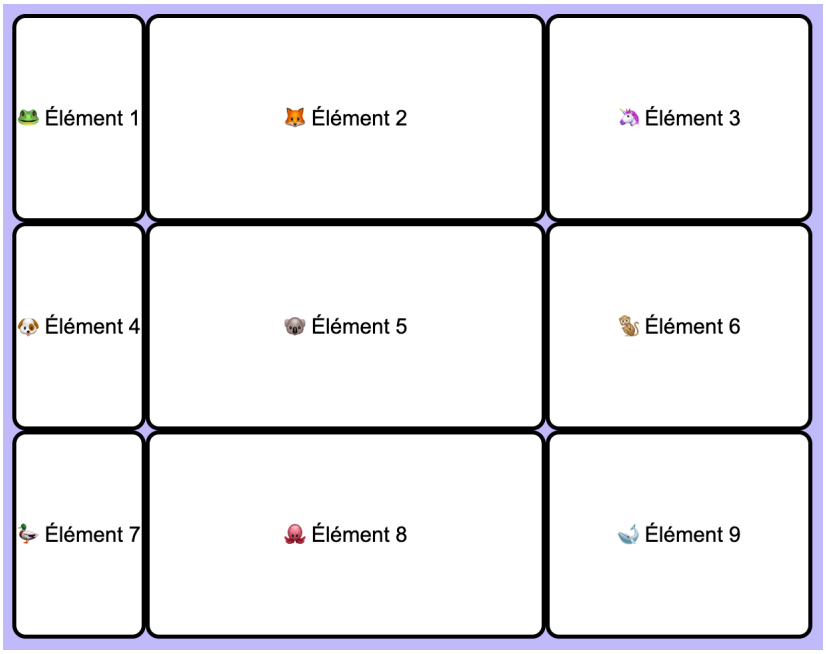
Magique, n'est-ce pas ?

On n'a même pas eu à dire « je veux 3 colonnes » : le nombre de colonnes a tout simplement été déduit par le navigateur, à partir du moment où on a spécifié 3 largeurs distinctes Si vous définissez 10 largeurs, vous aurez 10 colonnes.

Et vous pouvez moduler la largeur que vous souhaitez pour chacune de vos colonnes. Par exemple, on pourrait écrire :

```css
.conteneur {
    display: grid;
    grid-template-columns: 100px 300px 200px;
}
```

☞



2.1.2 Définissez vos rangées avec la propriété CSS grid-template-rows

Passons maintenant à la propriété `grid-template-rows`. Et on fait bien attention à supprimer la hauteur fixe qu'on avait associée à la classe `.box` : `height: 150px;`.


Pourquoi il faut faire ça ? Je ne comprends pas.

En fait, indiquer une hauteur avec la propriété `height` à `.box` a pour effet d'assigner une hauteur fixe pour tous les éléments de classe `.box`. Or, on peut avoir besoin de définir des hauteurs différentes aux rangées, ce qu'on va pouvoir spécifier avec `grid-template-rows`, en indiquant la hauteur de chacune, comme on l'a fait pour `grid-template-columns` :

```
.conteneur {
    display: grid;
    grid-template-columns: 200px 200px 200px;
    grid-template-rows: 100px 150px 200px;
}
```

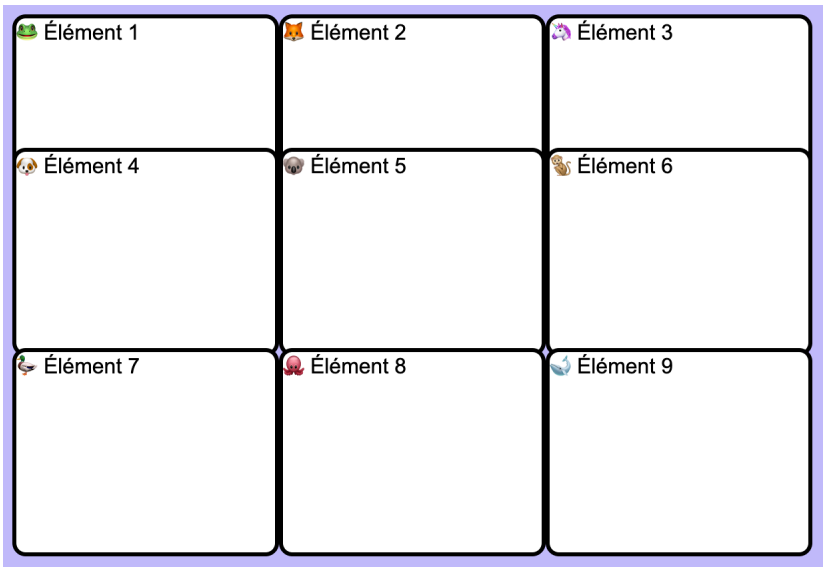
Si vous laissez la hauteur fixe et que vous indiquez en même temps différentes hauteurs pour les rangées :

```
.box {
  height: 150px;
}

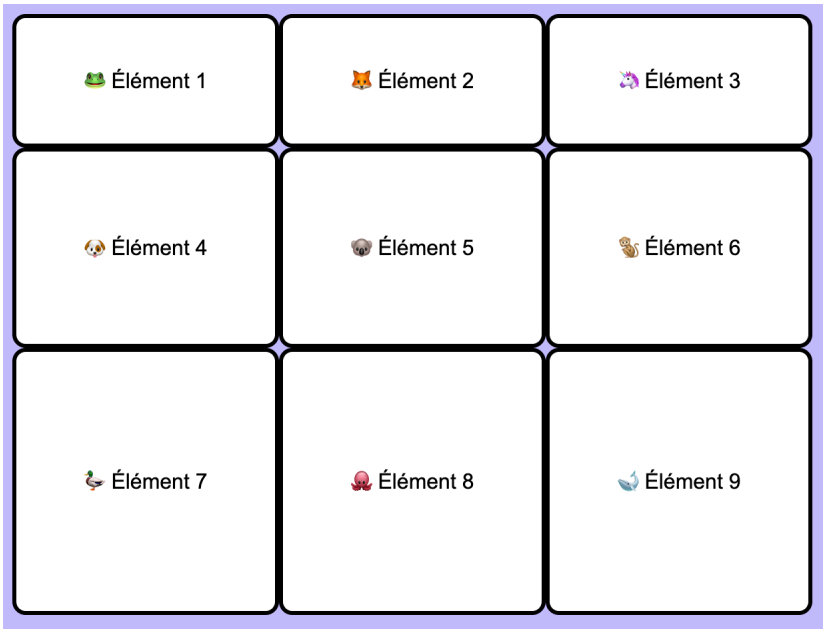
.conteneur {
    display: grid;
    grid-template-columns: 200px 200px 200px;
```

```
    grid-template-rows: 100px 150px 200px;
}
```

... ça ne marche pas, en tout cas, le navigateur va vouloir appliquer les deux instructions en même temps ce qui ne vous donnera pas l'effet voulu et les éléments vont se chevaucher :



Sans hauteur fixe et grâce aux hauteurs spécifiées une à une, on a donc bien 3 rangées, une de `100px` de hauteur, la seconde de `150px`, et la troisième de `200px` :





C'est un peu écrasé tout ça. Il faut que j'ajoute des marges ou paddings sur chaque élément pour faire respirer mon contenu ?

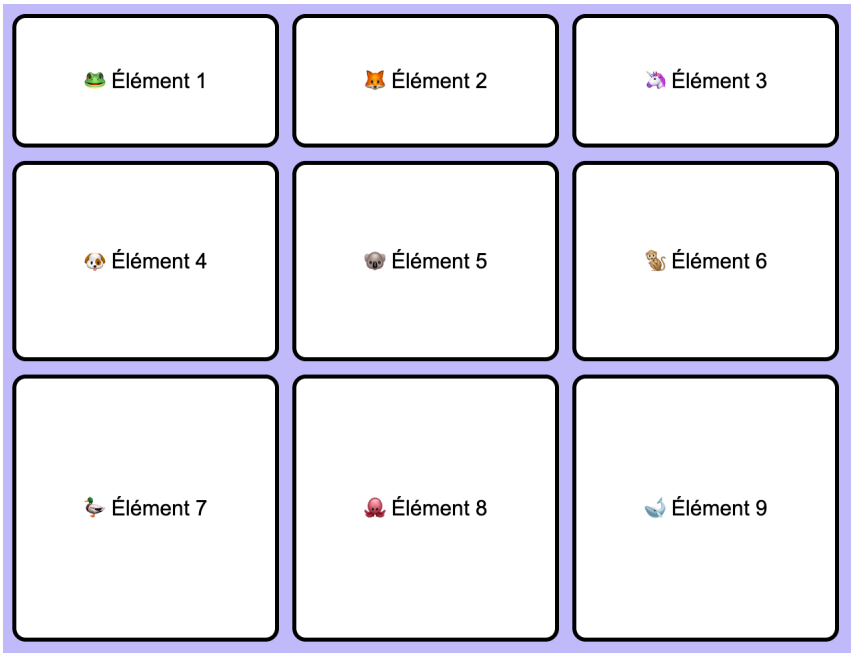
2.2 Aérez votre contenu avec la propriété CSS gap

La propriété CSS `gap` permet de créer des espacements entre vos éléments. Si vous voulez garder les mêmes distances entre les rangées et les colonnes sans avoir à vous compliquer la vie, vous précisez simplement une valeur.

On écrit alors :

```
.conteneur {
    display: grid;
    grid-template-columns: 200px 200px 200px;
    grid-template-rows: 100px 150px 200px;
    gap: 10px;
}
```

Et on obtient :



Les éléments sont espacés de 10 pixels

2.3 Choisissez vos unités

Jusqu'à maintenant, nous avons utilisé une unité en `px` pour définir les hauteurs et largeurs des rangées et colonnes. Mais il est également possible d'utiliser des `em`, `rem`, et des pourcentages.

Et encore mieux : je vais vous montrer une unité qui est spécifique aux grids : les "fraction units", ou `fr` pour les intimes, qui représentent des parts de largeur dans les colonnes.



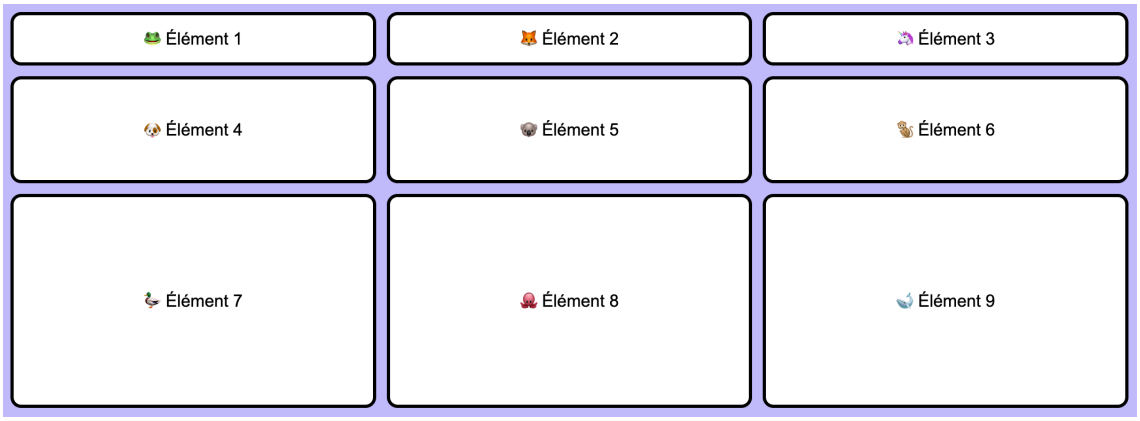
Mais pourquoi utiliser les `fr` alors qu'on a des pourcentages ?

Si vous tentez d'utiliser des pourcentages avec CSS Grid, vous verrez que ce n'est pas toujours simple. Si par exemple, vous souhaitez créer 2 colonnes dans une `grid`, et que vous mettez `50%` de largeur pour chacune d'entre elles, vous vous attendez sûrement à ce que votre grid prenne toute la largeur... Et pourtant non : les grid gaps (les espacements entre vos grids) peuvent venir mettre le bazar dans tout ça, et votre grid va dépasser. C'est pourquoi il est encore plus simple d'utiliser des `fr`, l'unité créée spécialement pour les CSS Grids.

Donc si dans notre code on a :

```
.conteneur {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: 50px 100px 200px;
    gap: 10px;
}
```

... on aura :

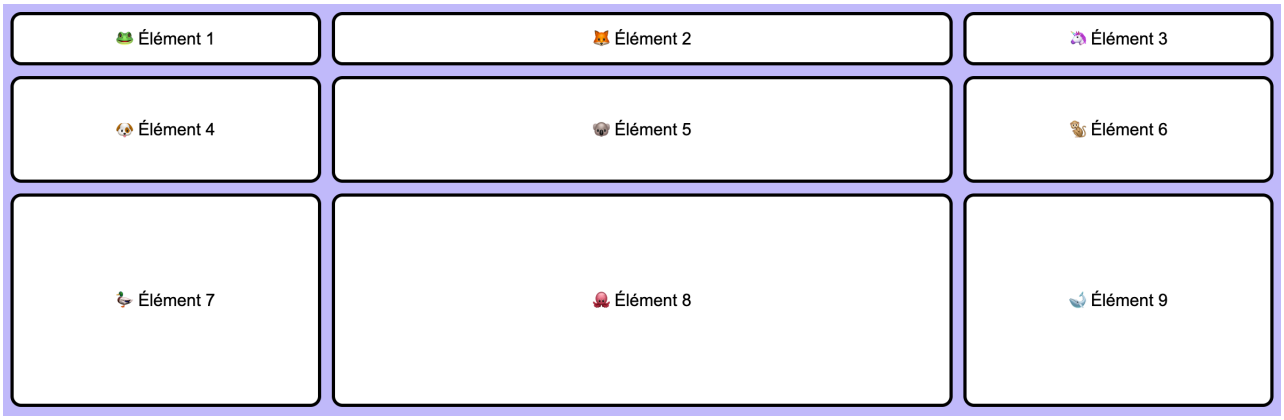


Les 3 colonnes ont la même largeur

... et vous pouvez ajuster la taille en fonction de la colonne.
Exemple :

```
.conteneur {
    display: grid;
    grid-template-columns: 1fr 2fr 1fr;
    grid-template-rows: 50px 100px 200px;
    grid-gap: 10px;
}
```

☞

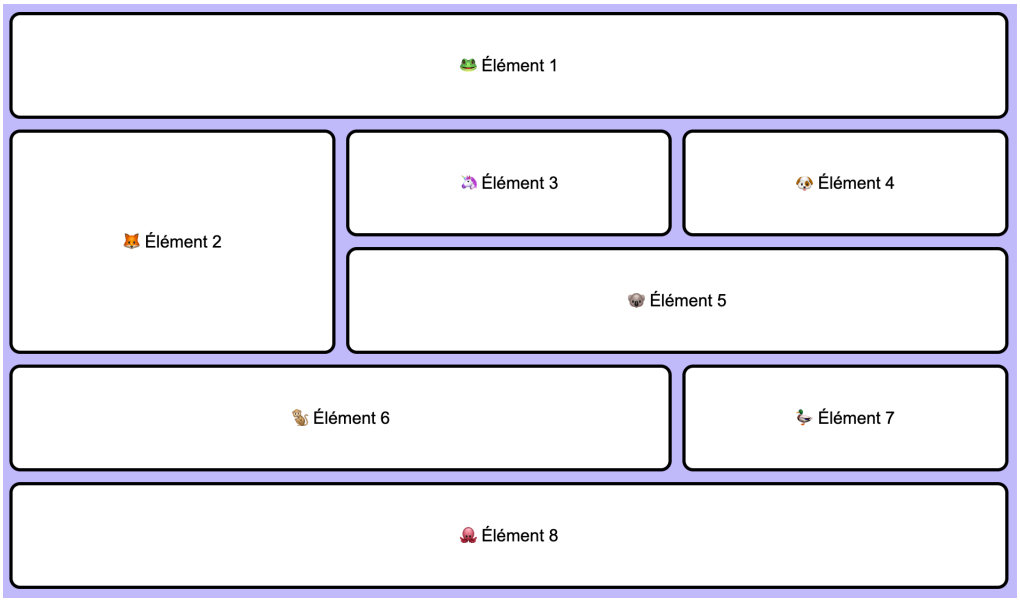


La deuxième colonne fait le double de la largeur des première et troisième colonnes

2.4 Définissez la taille des éléments de votre grid

Pour l'instant, nos grilles sont assez sages, et nous n'avons rien fait que nous ne pourrions faire avec Flexbox. Mais je vais vous montrer comment secouer tout ça !

Si vous vouliez obtenir ce résultat, par exemple ?



Pour cela, vous allez apprendre à maîtriser les propriétés liées à `grid-column` et `grid-row`.



C'est surtout dans ce genre de cas qu'on voit que CSS Grid est complémentaire à Flexbox. Créer ce type de mise en page serait vraiment plus laborieux avec Flexbox, alors qu'ici, on peut le faire assez simplement.

À chaque fois que vous écrivez `display: grid`, le navigateur se représente votre conteneur comme un ensemble d'éléments délimités par des lignes horizontales et verticales. Ces lignes sont invisibles, mais peuvent être inspectées avec les outils de développement.

Dans notre cas, pour réaliser une grid similaire à ce que je vous montrais, avec des éléments à cheval sur plusieurs colonnes ou plusieurs rangées, on va devoir indiquer un point de départ et un point d'arrivée pour nos éléments.

On aura besoin des propriétés suivantes pour déclarer nos éléments :

- `grid-column-start` indique la ligne verticale de départ de l'élément ;
- `grid-column-end` indique la ligne verticale d'arrivée de l'élément ;
- `grid-row-start` indique la ligne horizontale de départ de l'élément ;
- `grid-row-end` indique la ligne horizontale d'arrivée de l'élément.

Ainsi, on change un peu notre code pour avoir une structure qui nous permet simplement de mettre en place la grid. Ce qui nous donne en HTML :

```
<div class="conteneur">
    <div class="box une">Élément 1</div>
    <div class="box deux">Élément 2</div>
    <div class="box trois">Élément 3</div>
    <div class="box quatre">Élément 4</div>
    <div class="box cinq">Élément 5</div>
    <div class="box six">Élément 6</div>
    <div class="box sept">Élément 7</div>
    <div class="box huit">Élément 8</div>
    <div class="box neuf">Élément 9</div>
</div>
}
```

Et en CSS, on a :

```
.conteneur {
    display: grid;
    grid-template-columns: 1fr 1fr 1fr;
    grid-template-rows: 100px 100px 100px 100px 100px;
    grid-gap: 10px;
}
```

2.5 Mesurez vos colonnes

Prenons notre première `div` : `.une`.

Si on veut qu'elle prenne toute la largeur, comme dans le résultat attendu, on doit alors préciser :

```
.une {
    grid-column-start: 1;
    grid-column-end: 4;
}
```

Et vous pouvez même l'écrire en une seule ligne avec la propriété raccourcie `grid-column` :

```
.une {
    grid-column: 1 / 4;
}
```



Attendez, mais pourquoi on dit d'aller jusqu'à `4` pour prendre toute la largeur, alors qu'il n'y a que 3 colonnes ? On ne devrait pas plutôt dire `grid-column-end: 3;` pour dire qu'elle s'arrête à la troisième colonne ?

Et non car ça reviendrait à dire qu'elle s'arrête juste avant le dernier tiers, donc juste avant la troisième colonne. C'est déroutant je vous l'accorde mais en fait on compte +1 avec cette technique. Le navigateur considère que pour couvrir toute la largeur, on "atteint" un élément suivant.

2.6 Mesurez vos rangées

On veut maintenant créer le deuxième élément de la grid, qui s'étale sur deux rangées en hauteur, à partir de la deuxième rangée (l'élément `.une` occupe déjà la première rangée). On précise donc que `.deux` commence à la rangée 2 et finit sur la rangée 4 :

```
.deux {
    grid-row-start: 2;
    grid-row-end: 4;
}
```

Ce qui équivaut à :

```
.deux {
    grid-row: 2 / 4;
}
```

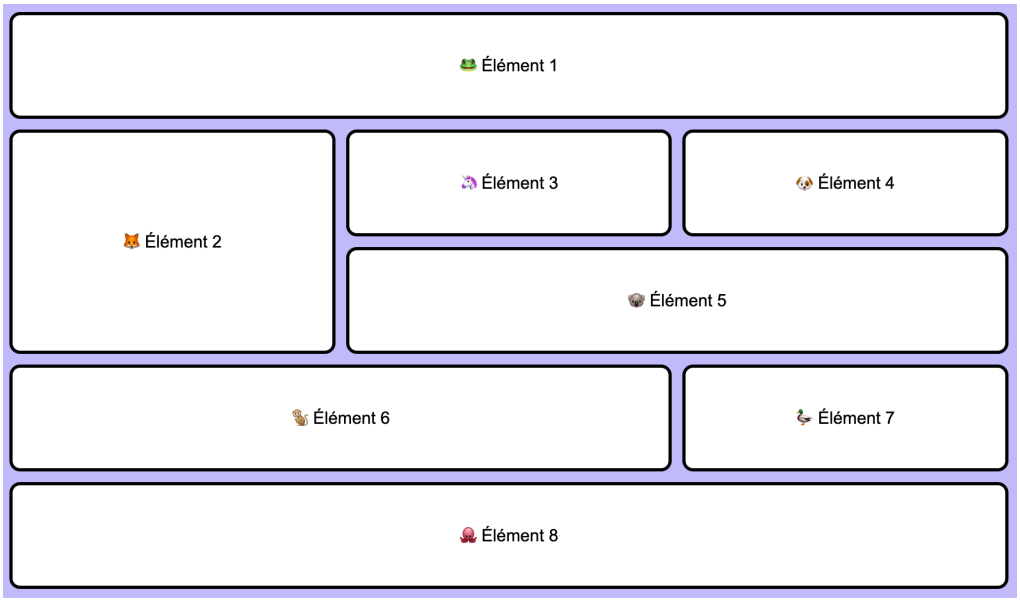
En allant jusqu'au bout de notre logique, on obtient pour les éléments suivants :

```
.cinq {
  grid-column: 2 / 4;
  grid-row: 3 / 4;
}

.six {
  grid-column: 1 / 3;
  grid-row: 4 / 5;
}
```

```css
.huit {
  grid-column: 1 / 4;
}
```

☞



Si vous avez l'impression de vous embrouiller entre les différentes propriétés de CSS Grid, pas de panique. CSS Grid est relativement avancé en CSS. L'idée de ce chapitre est de vous montrer le champ des possibles avec les grids.

Si vous voulez aller plus loin, vous pouvez :

- creuser la documentation de la mise en page à l'aide des CSS Grids ;
- suivre le cours Create Web Page Layouts with CSS (il n'existe pas en version française sur la plateforme pour l'instant) ;
- vous faire la main sur le petit jeu Grid Garden (en anglais) (vous pouvez créer un compte gratuitement sur Codepip).

Nous avons vu quelques-unes des propriétés les plus utiles des CSS Grids, mais il y en a bien d'autres. Des raccourcis qui vous simplifient encore plus la vie, des syntaxes particulièrement claires. Donc n'hésitez pas à jeter un œil à tout cela un peu plus tard dans votre apprentissage du CSS.

2.7 Travail à faire



Vous trouverez une nouvelle page portfolio. La base de la page a déjà été codée. De nouvelles images ont également été ajoutées dans `images/portfolio`.



À partir de là, vous devrez coder deux grids :

- une première grid avec les images dont le nom commence par "paysage". Elle comprend 3 colonnes et 3 rangées ;
- une deuxième grid de 3 colonnes et 2 rangées, avec les images dont le nom commence par "portrait".

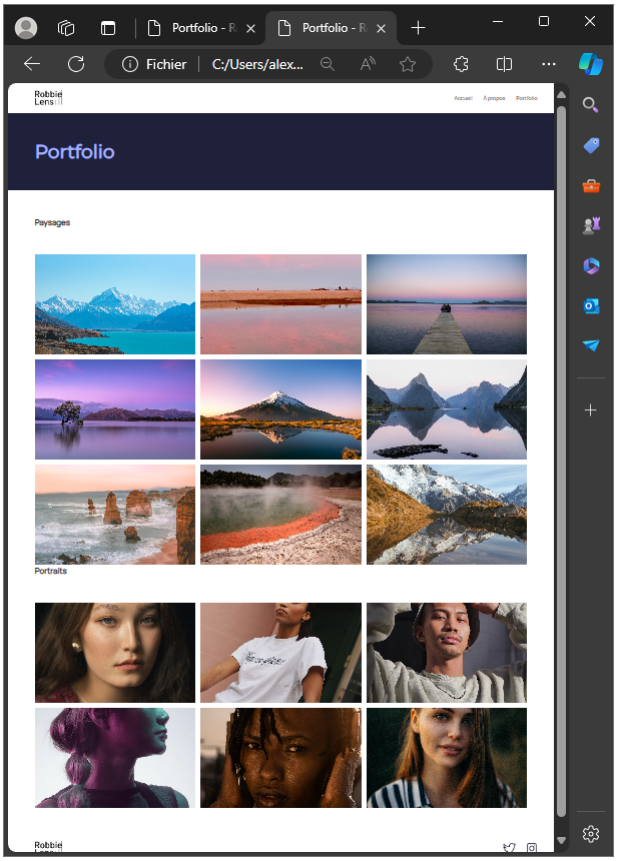
Les colonnes devront s'adapter à la largeur de l'écran, et les rangées feront `300px` de haut, avec un écart de `15px`.

Même si nous ne verrons pas ces propriétés dans ce cours, vous aurez besoin du bout de code suivant dans les sélecteurs `.grid-paysages img` et `.grid-portraits img` afin que la taille de vos images s'adaptent à la taille de votre grid :

```
.grid-paysages img,
.grid-portraits img {
  /* Même si vous ne connaissez pas cette propriété, vous en aurez besoin
  pour permettre aux images de respecter les dimensions de la grid CSS */
  object-fit: cover;
}
```

Les images des deux grids font la même taille.

Voici un aperçu du résultat attendu :



2.8 Synthèse

- Les CSS Grids sont complémentaires à Flexbox et permettent de créer facilement des mises en page plus élaborées que Flexbox, sans forcément avoir des éléments de la même taille.

- Pour déclarer une grid, on déclare simplement `display: grid;` sur le conteneur : notre navigateur comprend tout de suite que nos éléments sont dans la grid.

- On définit les colonnes avec `grid-template-columns` et les rangées avec `grid-template-rows` : en fonction du nombre de valeurs passées, de nouvelles colonnes et rangées sont créées.

- En plus des unités classiques `px`, `em`, `rem` et `%`, les `fr` sont encore plus simples, et permettent d'indiquer une fraction de la grille.

- `gap` permet d'espacer les éléments entre eux.

- Les grids créent implicitement des lignes horizontales et verticales délimitant les différentes rangées et colonnes.

- Chaque élément peut avoir :

  son propre point de départ horizontal avec `grid-row-start` ;
  son point d'arrivée horizontal avec `grid-row-end` ;

son point de départ vertical avec `grid-column-start` ;
et son point d'arrivée vertical avec `grid-column-end`.

Pour ceux qui pédalent vite

3 Abordez d'autres techniques de mise en page

3.1 Transformez vos éléments avec display

La propriété CSS `display` permet de transformer n'importe quel élément de votre page, d'un type vers un autre. Avec cette propriété, on peut par exemple imposer à des liens (originellement de type `inline`) d'apparaître sous forme de type `block` :

```
a {
    display: block;
}
```

À ce moment-là, les liens vont se positionner les uns en dessous des autres (comme des blocs normaux), et il devient possible de modifier leurs dimensions ! Et à l'inverse, vous pouvez tenter de faire en sorte qu'un élément qui est de type `block` par défaut devienne `inline`.

3.2 Cachez vos éléments avec display : none ;

On peut également décider de masquer complètement un élément de la page en attribuant à display la valeur none. Par exemple, si je veux masquer les éléments qui ont la classe secret, je vais écrire :

```
.secret {
    display: none;
}
```

none est très pratique lorsqu'il faut cacher certains éléments en fonction de la taille de l'écran.

La liste des différentes valeurs possibles pour `display` est longue. Nous n'aurons pas le temps de toutes les aborder ensemble ici, mais sachez qu'il est notamment possible de tirer parti du meilleur de `inline` et `block`. Pour cela, vous pouvez utiliser `display: inline-block;` qui rend vos éléments `inline` tout en permettant de les dimensionner.

3.3 Positionnez vos éléments avec la propriété CSS position

Dans les méthodes que nous avons vues jusqu'à maintenant, le navigateur dispose les éléments afin qu'ils ne se superposent jamais. Ils font partie d'un flux normal. Mais nous allons maintenant voir une nouvelle propriété : `position`. Elle permet de positionner avec précision des éléments sur la page (et même parfois de les superposer).

- Le **positionnement relatif** permet de décaler l'élément par rapport à sa position normale.
- Le **positionnement absolu** permet de placer un élément n'importe où sur la page (en haut à gauche, en bas à droite, tout au centre, etc.).
- D'autres **types de positionnements** comme **fixed** ou encore **sticky**, qui ont des comportements assez semblables, peuvent être utiles si l'on veut qu'un élément, tel que le menu de

navigation par exemple, reste à l'écran.

Le positionnement fonctionne aussi bien sur des balises qui sont `inline` que `block`. Mais dans les faits, vous verrez qu'on l'utilise bien plus souvent sur des balises `block` que sur des balises `inline`.

Il faut d'abord faire son choix entre les trois modes de positionnement disponibles. Pour cela, on utilise la propriété CSS `position`.

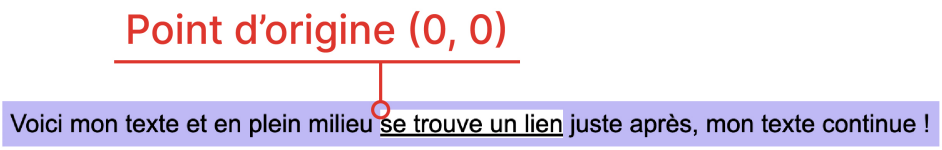
Regardons tout ça d'un peu plus près !

3.3.1 Définissez un positionnement relatif avec position : relative ;

Le positionnement relatif permet d'effectuer des ajustements : l'élément est décalé par rapport à sa position initiale.

Prenons par exemple un texte dans lequel se trouve un lien. Pour commencer, on donne une couleur de fond au body, et on met le lien sur un fond blanc.

Ici, on utilise `position: relative;`. Notre élément a alors un point d'origine : le point de coordonnées `(0, 0)` qui va nous permettre de le déplacer :



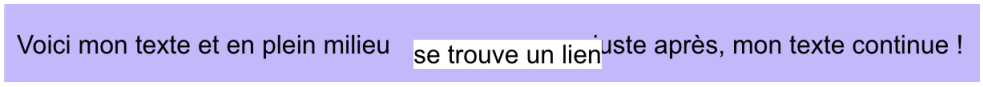
On accède alors à quatre propriétés CSS qui vont nous permettre de décaler notre élément :

- `left` : décalage depuis la gauche.
- `right` : décalage depuis la droite.
- `top` : décalage depuis le haut de notre élément.
- `bottom` : décalage depuis le bas de notre élément.

Ainsi, si j'écris le CSS suivant :

```
a {
    background-color: white;
    position: relative;
    top: 6px;
    left: 10px;
}
```

Regardez ce que ça donne :



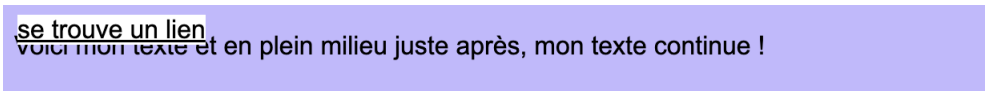
L'élément est décalé par rapport au point d'origine

3.3.2 Définissez un positionnement absolu avec position : absolute ;

Le positionnement absolu permet de placer un élément (réellement) n'importe où sur la page. Pour l'utiliser, on écrit tout simplement `position: absolute;` :

```
a {
    background-color: white;
    position: absolute;
    top: 6px;
    left: 10px;
}
```

☞



Le lien est décalé par rapport à la page

3.3.3 A votre tour

Prenons un autre exemple maintenant.



Code HTML :

```
<p>
  Voici mon premier paragraphe
</p>

<p>
  Puis mon second <a href="">au milieu duquel on a mis un lien</a> puis on continue le
</p>

<p>Et enfin, un troisième paragraphe</p>

<div>
  <div class="box rebelle">Élément 1</div>
  <div class="box">Élément 2</div>
  <div class="box">Élément 3</div>
<div>
```

Code CSS :

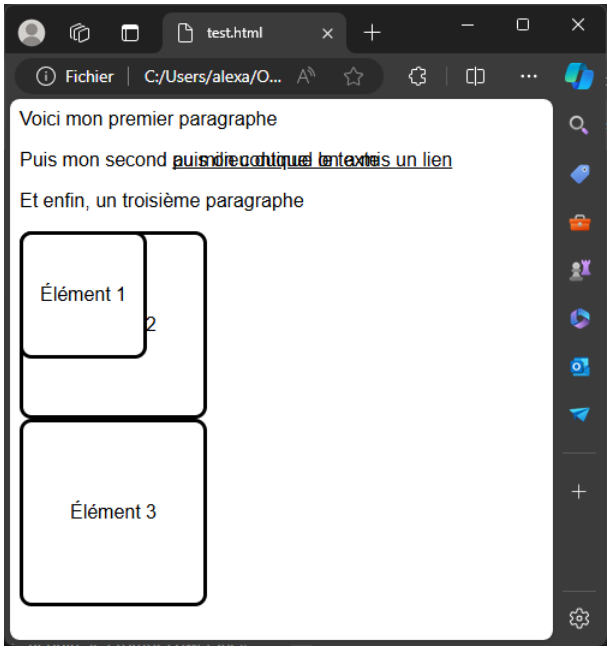
```css
* {
  font-family: Arial;
}

.box {
  background-color: #fff;
  border-radius: 10px;
  border: 3px solid black;
  height: 150px;
  width: 150px;
  display: flex;
  justify-content: center;
  align-items: center;
}

a {
  color: black;
  position: absolute;
}

.rebelle {
  position: absolute;
  height: 100px;
  width: 100px;
}
```

Résultat :



Bon c'est bien beau, mais cela ne suffit pas pour positionner un élément où on souhaite vraiment. Pour y parvenir, on va refaire appel à nos `left`, `right`, `top` et `bottom`.

On pourra leur donner une valeur en pixels, comme `14px` ou une valeur en pourcentage, comme `50%`.

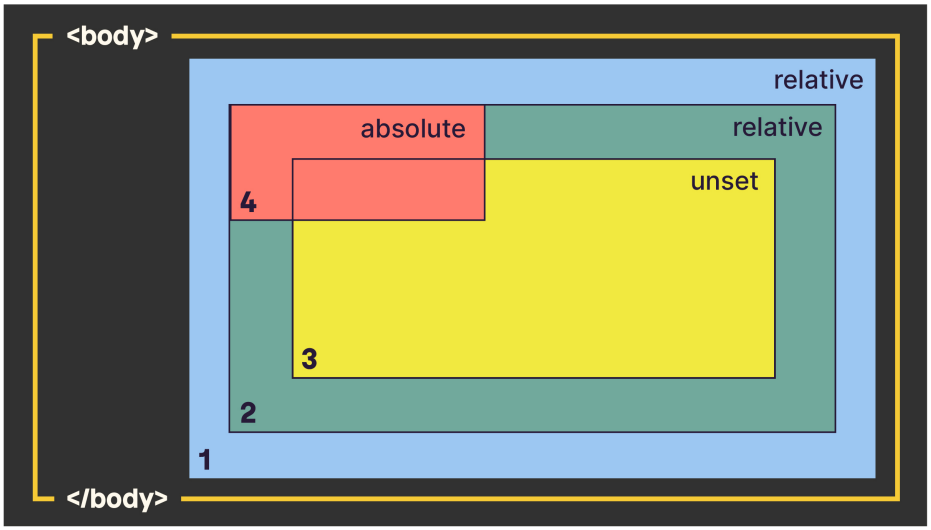
Habituellement, les éléments `inline` et `block` sont disposés dans le flux normal. Mais ici, vous voyez, nos éléments ne respectent plus rien, et sortent du flux.



Donc quand j'utilise `left`, `right`, `top` et `bottom`, mon élément va toujours se placer par rapport aux bords de la page ?

Eh bien... pas nécessairement. Un élément `absolute` va se positionner par rapport au premier élément qu'il rencontre dans ses parents, et qui utilise lui-même la propriété `position`. Donc si on veut placer un élément par-dessus un autre, il faudra que ce premier élément utilise aussi la propriété `position`.

Regardez un peu ce que cela donne :



Différentes valeur de la propriété position



La propriété position unset réinitialise la propriété afin que sa valeur soit la valeur héritée depuis l'élément parent, ou soit la valeur initiale (s'il n'y a pas d'héritage).

3.4 Gérez le chevauchement avec la propriété z-index

Vous l'avez remarqué, les éléments positionnés en absolu sont placés par-dessus le reste des éléments de la page. Mais si vous placez deux éléments en absolu aux mêmes coordonnées... ils vont se chevaucher.

Dans ce cas, vous pouvez utiliser la propriété `z-index` pour indiquer quel élément doit apparaître au-dessus des autres : vous empilez vos éléments dans l'ordre souhaité ! L'élément ayant la valeur de `z-index` la plus élevée sera placé par-dessus les autres.

Par exemple, si j'écris le HTML :

```
<div class="conteneur">
    <div class="box une">Élément 1</div>
    <div class="box deux">Élément 2</div>
    <div class="box trois">Élément 3</div>
<div>
```

... associé au CSS suivant :

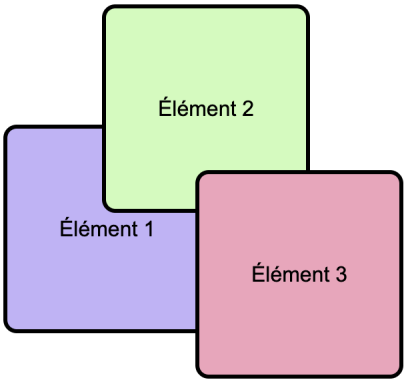
```
.conteneur {
    position: relative;
    height: 300px;
    width: 300px;
}

.une {
    background-color: #C2B0F9;
    position: absolute;
    left: 0;
    top: 90px;
}

.deux {
    background-color: #CBFCB9;
    position: absolute;
    right: 70px;
    top: 0;
}

.trois {
    background-color: #F2A3BB;
    position: absolute;
    right: 0;
    bottom: 20px;
}
```

... on obtient :



3.5 Bloquez un élément avec fixed ou sticky

Le principe est exactement le même que pour le positionnement absolu sauf que, cette fois, le bloc se fige, même si on descend plus bas dans la page. Pour faire cela, il y a deux options possibles : attribuer la valeur `fixed` ou `sticky` à la propriété CSS `position` :

3.5.1 A votre tour



Vous allez voir tout de suite la différence avec le code suivant lorsque vous essaierez de scroller pour descendre plus bas sur la page !

Code HTML :

```
<h1>Scrollez dans la suite de la page pour voir</h1>
<div class="conteneur">
  <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat
  non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
  </p>
  <div class="box sticky">Élément sticky</div>
  <div class="box fixed">Élément fixed</div>
  <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod
  tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam,
  quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo
  consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse
  cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat
  non proident, sunt in culpa qui officia deserunt mollit anim id est
  laborum.</p>
  <p>Sed ut perspiciatis unde omnis iste natus error sit voluptatem
  accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab
  illo inventore veritatis et quasi architecto beatae vitae dicta sunt
  explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut
  odit aut fugit, sed quia consequuntur magni dolores eos qui ratione
  voluptatem sequi nesciunt. Neque porro quisquam est, qui dolorem ipsum quia
  dolor sit amet, consectetur, adipisci velit, sed quia non numquam eius modi
  tempora incidunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut
  enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit
  laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum
  iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae
  consequatur, vel illum qui dolorem eum fugiat quo voluptas nulla pariatur?
  </p>
<div>
```

Code CSS :

```css
* {
  font-family: Arial;
}

h1 {
  text-align: center;
}

.box {
  background-color: #fff;
  border-radius: 10px;
  border: 3px solid black;
  height: 150px;
  width: 150px;
  display: flex;
  justify-content: center;
  align-items: center;
}

.conteneur {
  position: relative;
  width: 50%;
  margin: auto;
}

.sticky {
  position: sticky;
  background-color: #C2B0F9;
  margin: auto;
  top: 0;
}

.fixed {
  position: fixed;
  background-color: #CBFCB9;
  top: 0;
  left: 0;
}
```

☞



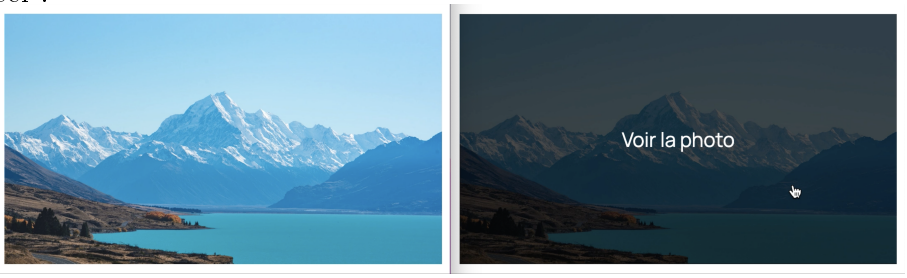
3.6 Travail à faire

Vous allez maintenant pouvoir exploiter ce que vous venez d'apprendre dans le portfolio de Robbie Lens.

Pour ce chapitre, vous allez :

- Les images de la page "Portfolio" peuvent maintenant s'afficher en plein écran avec la méthode vue dans le chapitre "Insérez des images". Pour cela, des liens vont être ajoutés avec la classe `lien-conteneur-photo`.
- Transformer les liens qui ont la classe `.cta` en éléments `inline-block` avec `display` sur les pages "Accueil" et "À propos", et ajouter leur lien vers la page "Portfolio" ; (cta veut dire call to action en anglais).
- ajouter des `margin` et `padding` là où il en manque sur les pages "Accueil" et "À propos".
- créer sur les images un effet au survol de la souris en affichant un élément (grâce à `display`) qui :

  aura un fond noir d'une opacité de `70%`;
  s'affichera par-dessus l'image, et y mentionnera le texte suivant : "Voir la photo", comme ceci :



Avant et après le passage de la souris sur l'image

3.7 Synthèse

- La propriété `display` permet de changer le comportement de base d'un élément :

transformer un élément `inline` en `block` ;
inversement, un élément `block` en `inline` ;
mais aussi de faire un mélange des deux avec `inline-block`.

- Le positionnement relatif permet de décaler un bloc par rapport à sa position normale.

- Le positionnement absolu permet de placer un élément où l'on souhaite sur la page, au pixel près.

- Un élément A positionné en absolu à l'intérieur d'un autre élément B (lui-même positionné en absolu, fixe ou relatif) se positionne par rapport à l'élément B, et non par rapport au coin en haut à gauche de la page.

- Lorsque plusieurs éléments s'empilent, il est possible de les ordonner avec `z-index`.